

ConnectCall

Project Documentation & Setup Guide

GitHub Repository: <https://github.com/Aryyl/ConnectCall>
Download APK: <https://github.com/Aryyl/ConnectCall/releases/tag/v1.0.0>
Video Demo & APK (Drive): https://drive.google.com/drive/folders/1oIytxh9vEb20_-0sKnxacdoVvrGlHST7?usp=drive_link

ConnectCall is a modern, real-time audio and video calling Flutter application. It features a sleek Material 3 user interface, persistent background calling capabilities, and robust user management. The app is designed to deliver a seamless communication experience with offline push notifications, allowing devices to ring even when the app is asleep or killed.

Features

- **Real-Time Communication:** 1-on-1 and Group Audio/Video calling.
- **Offline Push Notifications:** Calls ring on the receiver's device even if the app is killed, using ZPNs (ZEGOCLOUD Push Notification service).
- **Screen Sharing:** (*Currently experiencing issues*) Built-in screen sharing capabilities utilizing the gallery layout.
- **User Authentication:** Secure user authentication managed via Firebase Auth.
- **User Management & Blocking:** A robust user directory allowing you to block/unblock users to prevent unwanted calls.
- **Call History:** Automatic logging of incoming, outgoing, missed, and rejected calls synced in real-time.
- **Automated Permissions:** Clean, sequential permission request flow (Camera, Mic, Notifications, Display Over Other Apps) on first launch to ensure smooth call connections.

Tech Stack & Architecture

- **Flutter Version:** Flutter 3.24+
- **Architecture:** Feature-first modular architecture utilizing Controller/Service patterns.
- **State Management:** `flutter_riverpod` (v2) for reactive, compile-safe state management.
- **Routing:** `go_router` featuring a persistent bottom navigation shell (`StatefulShellRoute`).
- **Backend:** Firebase
 - *Firebase Auth* for identity.
 - *Cloud Firestore* for user directories, blocking logic, and call history.
- **Calling SDK:** ZEGOCLOUD

- `zego_uikit_prebuilt_call` for the core calling UI and WebRTC engine.
- `zego_uikit_signaling_plugin` & `zego_zpns` for offline call invitations and push notifications.

Key Packages Used

- `firebase_core`, `firebase_auth`, `cloud_firestore`
- `flutter_riverpod`, `riverpod_annotation`
- `go_router`
- `zego_uikit_prebuilt_call`, `zego_uikit_signaling_plugin`, `zego_zpns`
- `permission_handler`
- `flutter_dotenv`
- `uuid`

Setup Instructions

Prerequisites

1. **Flutter SDK** installed and configured on your machine.
2. A **Firebase Project** with Authentication (Email/Password) and Firestore enabled.
3. A **ZEGOCLOUD account** with a registered project to obtain your App ID and App Sign.

1. Clone & Install Dependencies

```
git clone <repository_url>
cd connect_call
flutter pub get
```

2. Firebase Configuration

- Download your `google-services.json` from the Firebase Console.
- Place it inside the `android/app/` directory.
- *(For iOS, place the `GoogleService-Info.plist` inside the `ios/Runner/` directory).*

3. Environment Variables (.env)

The app uses `flutter_dotenv` to securely manage ZEGOCLOUD credentials. Create a `.env` file in the root directory (`connect_call/.env`) and add your ZEGOCLOUD credentials:

```
ZEGO_APP_ID=1234567890
ZEGO_APP_SIGN=your_64_character_hex_string_here
```

Ensure that `.env` is declared in your `pubspec.yaml` under the `assets:` section.

4. Run the App

To test audio/video capabilities and offline push notifications, it is highly recommended to run the app on **physical devices** rather than emulators.

```
flutter run
```

Known Limitations

1. **Simulator/Emulator Constraints:** Audio, Video, and Push Notifications behave unreliably on emulators. Always test on physical devices.
2. **iOS Background Calling:** Offline push notifications on iOS require additional configuration (APNs certificates) in the ZEGOCLOUD console and Apple Developer Portal.
3. **Android 14+ Permissions:** The app requests `USE_FULL_SCREEN_INTENT` and `SYSTEM_ALERT_WINDOW` explicitly. Depending on the Android manufacturer (e.g., Xiaomi, Vivo), the user may still need to manually allow "Display pop-up windows while running in the background" in the OS settings.
4. **Network Sensitivity & Stability:** The underlying calling technology is highly sensitive to network fluctuations. Poor or unstable internet connections can cause calling screen glitches, delayed connections, or dropped calls.

AI Tools Used

- Antigravity